

JSON — Data আদান-প্রদান ও API Communication-এর সবচেয়ে গুরুত্বপূর্ণ Format

আধুনিক ওয়েব ডেভেলপমেন্টে ফ্রন্টএন্ড ও ব্যাকএন্ডের মধ্যে ডেটা আদান-প্রদান, ক্লায়েন্ট-সাইড স্টোরেজে অবজেক্ট পারসিস্ট করা এবং বিভিন্ন সিস্টেমের মধ্যে যোগাযোগ স্থাপনের জন্য **JSON (JavaScript Object Notation)** হলো সবচেয়ে হালকা, দ্রুত এবং বিশ্বজনীন মানদণ্ড।

JS Object → JSON.stringify() → JSON String (Payload) → Network / Storage → JSON.parse() → JS Object

ফুলস্ট্যাক ও আধুনিক ফ্রন্টএন্ড ইঞ্জিনিয়ারিংয়ে এপিআই কমিউনিকেশনের ভিত্তিই হলো এই JSON আর্কিটেকচার। 🔥

PART 01 — JSON FUNDAMENTALS, SYNTAX & DATA TYPES (19.1 – 19.14)

19.1 JSON কী?

JSON = JavaScript Object Notation

JSON হলো এমন একটি **text-based data format**, যেটা বিভিন্ন application-এর মধ্যে data আদান-প্রদানের জন্য ব্যাপকভাবে ব্যবহৃত হয়।

সহজভাবে:

JSON হলো structured data লেখার একটি standard format।

Example:

19.1 JSON

DATA FORMAT

```
{  
  "name": "Shariar",  
  "age": 24,  
  "role": "Developer"  
}
```

এটি দেখতে JavaScript object-এর মতো।

কিন্তু মনে রাখবে:

JSON নিজে JavaScript Object নয়! JSON হলো text format।

19.2 Real-life Example

ধরো Frontend থেকে Backend-এ user data পাঠাবে।

Frontend:

TEXT

Name: Shariar
Email: shariar@gmail.com
Age: 24

Backend-এ structured data পাঠানো যায়:

19.2 JSON

DATA FORMAT

```
{  
  "name": "Shariar",  
  "email": "shariar@gmail.com",  
  "age": 24  
}
```

Backend data process করে database-এ save করতে পারে।

19.3 কেন JSON এত গুরুত্বপূর্ণ?

Modern web development-এ:

```
Frontend  
↓  
HTTP Request  
↓  
API  
↓  
Backend  
↓  
Database
```

এখানে data আদান-প্রদানের জন্য JSON খুব common।

যেমন:

```
React  
↑  
REST API  
↑  
Node.js / Express  
↑  
Database
```

তুমি Full Stack Developer হিসেবে JSON প্রায় প্রতিদিনই ব্যবহার করবে।

19.4 JSON-এর Basic Structure

19.4 JSON

DATA FORMAT

```
{  
  "name": "Shariar",  
  "age": 24,  
  "student": true  
}
```

এখানে:

```
"name"    → Key  
"Shariar" → Value  
  
"age"     → Key  
24        → Value
```

অর্থাৎ:

TEXT

Key : Value

19.5 JSON Data Types

JSON-এ মূলত ৬ ধরনের data থাকে:

```
String
Number
Boolean
Object
Array
null
```

19.6 JSON String

19.6 JSON

DATA FORMAT

```
{
  "name": "Shariar"
}
```

String অবশ্যই double quote-এর মধ্যে হবে।

✓ CORRECT:

19.6 JSON

DATA FORMAT

```
{
  "name": "Shariar"
}
```

✗ INVALID JSON:

19.6 JSON

DATA FORMAT

```
{
  "name": 'Shariar'
}
```

JSON standard-এ string-এর জন্য double quotes ব্যবহার করতে হয় (Single Quote '...' ব্যবহার করলে SyntaxError হবে)।

19.7 JSON Number

19.7 JSON

DATA FORMAT

```
{
  "age": 24,
  "price": 70000,
  "rating": 4.5
}
```

Number-এর জন্য quote লাগে না।

19.8 JSON Boolean

19.8 JSON

DATA FORMAT

```
{
  "isStudent": true,
  "isActive": false
}
```

এখানে:

TEXT

```
true
false
```

quote-এর মধ্যে থাকবে না।

19.9 JSON `null`

19.9 JSON

DATA FORMAT

```
{
  "middleName": null
}
```

এর অর্থ এখানে বর্তমানে কোনো value নেই।

19.10 JSON Array

19.10 JSON

DATA FORMAT

```
{
  "skills": ["HTML", "CSS", "JavaScript", "React"]
}
```

Array-এর মধ্যে multiple values রাখা যায়।

19.11 JSON Object

Nested object:

19.11 JSON

DATA FORMAT

```
{
  "name": "Shariar",
  "address": {
    "city": "Dhaka",
    "country": "Bangladesh"
  }
}
```

এখানে `address` নিজেই একটি object।

19.12 Nested JSON

Real-world API response-এ nested data খুব common।

19.12 JSON

DATA FORMAT

```
{
  "user": {
    "id": 101,
    "name": "Shariar",
    "profile": {
      "role": "Developer",
      "skills": ["JavaScript", "React", "Node.js"]
    }
  }
}
```

JavaScript-এ:

19.12 JAVASCRIPT

RUN

```
console.log(data.user.name);
```

TEXT

Shariar

অর:

19.12 JAVASCRIPT

RUN

```
console.log(data.user.profile.skills[0]);
```

JavaScript

19.13 JSON বনাম JavaScript Object

JavaScript object:

19.13 JAVASCRIPT

RUN

```
const user = {  
  name: "Shariar",  
  age: 24,  
};
```

JSON:

19.13 JSON

DATA FORMAT

```
{  
  "name": "Shariar",  
  "age": 24  
}
```

দেখতে একই রকম হলেও পার্থক্য আছে।

JavaScript Object	JSON
JavaScript-এর data structure	Text-based data format
Key quote ছাড়া হতে পারে	Key double quote-এ থাকতে হবে
Single quote string হিসেবে ব্যবহার করা যায়	String-এর জন্য double quote
<code>undefined</code> থাকতে পারে	<code>undefined</code> নেই
Function থাকতে পারে	Function নেই
<code>Date</code> object থাকতে পারে	Native Date type নেই
Methods থাকতে পারে	Methods নেই
Directly executable data structure	Data representation

19.14 JSON কেন JavaScript Object নয়?

ধরো:

19.14 JAVASCRIPT

RUN

```
const user = {  
  name: "Shariar",  
};  
  
console.log(typeof user);
```

object

এখন:

19.14 JAVASCRIPT

RUN

```
const jsonData = '{"name":"Shariar"}';  
  
console.log(typeof jsonData);
```

TEXT

string

অর্থাৎ JSON representation সাধারণত JavaScript-এর কাছে **string** হিসেবে থাকে।

19.15 JSON.stringify()

JavaScript value → JSON string

এটি `JSON.stringify()`-এর কাজ।

19.15 JAVASCRIPT

RUN

```
const user = {
  name: "Shariar",
  age: 24,
};

const jsonData = JSON.stringify(user);

console.log(jsonData);
```

TEXT

```
{"name":"Shariar","age":24}
```

19.16 typeof JSON.stringify()

19.16 JAVASCRIPT

RUN

```
const user = {
  name: "Shariar",
  age: 24,
};

const jsonData = JSON.stringify(user);

console.log(typeof jsonData);
```

TEXT

```
string
```

19.17 JSON.parse()

JSON string → JavaScript value

19.17 JAVASCRIPT

RUN

```
const jsonData = '{"name":"Shariar","age":24}';

const user = JSON.parse(jsonData);

console.log(user);
```

19.17 JAVASCRIPT

RUN

```
{
  name: "Shariar",
  age: 24
}
```

19.18 JSON.parse() `Type`

19.18 JAVASCRIPT

RUN

```
console.log(typeof user);
```

```
object
```

19.19 Main JSON Formula

এই দুইটা খুব ভালোভাবে মনে রাখবে:

JavaScript → JSON

19.19 JAVASCRIPT

RUN

```
JSON.stringify(data);
```

JSON → JavaScript

19.19 JAVASCRIPT

RUN

```
JSON.parse(json);
```

Mental model:

```
JavaScript Object
  ↓
JSON.stringify()
  ↓
JSON String
```

Reverse:

```
JSON String
  ↓
JSON.parse()
  ↓
JavaScript Object
```

19.20 Array → JSON

19.20 JAVASCRIPT

RUN

```
const skills = ["HTML", "CSS", "JavaScript"];

const jsonSkills = JSON.stringify(skills);

console.log(jsonSkills);
```

```
["HTML","CSS","JavaScript"]
```

19.21 JSON → Array

19.21 JAVASCRIPT

RUN

```
const jsonSkills = '["HTML","CSS","JavaScript"]';

const skills = JSON.parse(jsonSkills);

console.log(skills);
```

```
["HTML", "CSS", "JavaScript"]
```

19.22 Nested Object → JSON

19.22 JAVASCRIPT

RUN

```
const student = {
  name: "Shariar",

  academic: {
    department: "CSE",
    semester: 8,
  },

  skills: ["JavaScript", "React", "Node.js"],
};

const jsonStudent = JSON.stringify(student);

console.log(jsonStudent);
```

```
{"name":"Shariar","academic":{"department":"CSE","semester":8},"skills":["JavaScript","React","Node.js"]}
```

19.23 JSON Pretty Print

Normally JSON এক লাইনে আসে:

19.23 JAVASCRIPT

RUN

```
JSON.stringify(student);
```

কিন্তু readable format চাইলে:

19.23 JAVASCRIPT

RUN

```
JSON.stringify(student, null, 2);
```

19.23 JSON

DATA FORMAT

```
{
  "name": "Shariar",
  "academic": {
    "department": "CSE",
    "semester": 8
  },
  "skills": ["JavaScript", "React", "Node.js"]
}
```

এখানে:

```
2 → indentation
```

দিচ্ছে।

PART 03 — ADVANCED SERIALIZATION: REPLACER, REVIVER & SERIALIZATION QUIRKS (19.24 – 19.36)

19.24 JSON.stringify() এর তিনটি Argument

Syntax:

19.24 JAVASCRIPT

RUN

```
JSON.stringify(value, replacer, space);
```

Example:

19.24 JAVASCRIPT

RUN

```
JSON.stringify(user, null, 2);
```

এখানে:

```
value    → user
replacer → null
space    → 2
```


19.25 replacer

replacer ব্যবহার করে কোন property JSON-এ থাকবে তা control করা যায়।

19.25 JAVASCRIPT

RUN

```
const user = {
  name: "Shariar",
  age: 24,
  password: "secret123",
};

const json = JSON.stringify(user, ["name", "age"]);

console.log(json);
```

19.25 JSON

DATA FORMAT

```
{ "name": "Shariar", "age": 24 }
```

password বাদ গেছে।

⚠ Sensitive data JSON-এ stringify না করাই ভালো যদি তার প্রয়োজন না থাকে।

19.26 Replacer Function

আরও advanced:

19.26 JAVASCRIPT

RUN

```
const user = {
  name: "Shariar",
  age: 24,
  password: "secret",
};

const json = JSON.stringify(user, function (key, value) {
  if (key === "password") {
    return undefined;
  }

  return value;
});

console.log(json);
```

19.26 JSON

DATA FORMAT

```
{ "name": "Shariar", "age": 24 }
```

19.27 JSON.parse() `Reviver`

`JSON.parse()` -এরও second argument আছে:

19.27 JAVASCRIPT

RUN

```
JSON.parse(text, reviver);
```

Example:

19.27 JAVASCRIPT

RUN

```
const json = `
  {
    "name": "Shariar",
    "age": 24
  }
`;

const user = JSON.parse(json, function (key, value) {
  if (key === "age") {
    return value + 1;
  }

  return value;
});

console.log(user);
```

19.27 JAVASCRIPT

RUN

```
{
  name: "Shariar",
  age: 25
}
```

19.28 JSON এ Function রাখা যায়?

না।

ধরো:

19.28 JAVASCRIPT

RUN

```
const user = {
  name: "Shariar",

  greet() {
    console.log("Hello");
  },
};

const json = JSON.stringify(user);

console.log(json);
```

19.28 JSON

DATA FORMAT

```
{ "name": "Shariar" }
```

`greet()` বাদ গেছে।

কারণ:

JSON data format; executable function রাখার format নয়।

19.29 undefined এবং JSON

19.29 JAVASCRIPT

RUN

```
const user = {  
  name: "Shariar",  
  age: undefined,  
};  
  
console.log(JSON.stringify(user));
```

19.29 JSON

DATA FORMAT

```
{ "name": "Shariar" }
```

Object-এর property হিসেবে `undefined` সাধারণত বাদ যায়।

19.30 Array-এর মধ্যে undefined

19.30 JAVASCRIPT

RUN

```
const data = ["HTML", undefined, "JavaScript"];  
  
console.log(JSON.stringify(data));
```

19.30 JSON

DATA FORMAT

```
["HTML", null, "JavaScript"]
```

অর্থাৎ array-এর ক্ষেত্রে `undefined` → `null` হয়ে যায়।

19.31 NaN এবং Infinity

19.31 JAVASCRIPT

RUN

```
const data = {  
  a: NaN,  
  b: Infinity,  
  c: -Infinity,  
};  
  
console.log(JSON.stringify(data));
```

19.31 JSON

DATA FORMAT

```
{  
  "a": null,  
  "b": null,  
  "c": null  
}
```

JSON-এ এগুলোর আলাদা standard number representation নেই।

19.32 Date এবং JSON

JavaScript:

19.32 JAVASCRIPT

RUN

```
const data = {
  createdAt: new Date(),
};

console.log(JSON.stringify(data));
```

Date সাধারণত ISO string representation-এ serialize হয়।

উদাহরণ:

19.32 JSON

DATA FORMAT

```
{
  "createdAt": "2026-09-16T09:55:00.000Z"
}
```

অর্থাৎ JSON-এ native `Date` type নেই।

19.33 JSON Parse করার সময় Date

JSON থেকে date parse করলে:

19.33 JAVASCRIPT

RUN

```
const data = JSON.parse(json);

console.log(typeof data.createdAt);
```

TEXT

string

এটি আবার Date object বানাতে:

19.33 JAVASCRIPT

RUN

```
const date = new Date(data.createdAt);

console.log(date);
```

19.34 Circular Reference

একটি object নিজেকেই reference করলে:

19.34 JAVASCRIPT

RUN

```
const user = {
  name: "Shariar",
};

user.self = user;
```

এখন:

19.34 JAVASCRIPT

RUN

```
JSON.stringify(user);
```

Error হবে:

```
TypeError:
Converting circular structure to JSON
```

কারণ JSON-এর normal object structure circular reference represent করতে পারে না।

19.35 JSON Parse Error

Invalid JSON parse করলে error হবে।

19.35 JAVASCRIPT

RUN

```
const json = '{"name":"Shariar"}';

try {
  const data = JSON.parse(json);

  console.log(data);
} catch (error) {
  console.error("Invalid JSON:", error.message);
}
```

Invalid JSON: ...

এখানে Chapter 14-এর `try ... catch` আবার কাজে লাগছে।

19.36 JSON-এর Common Syntax Rules

Rule 1 — Keys double quotes



19.36 JSON

DATA FORMAT

```
{  
  "name": "Shariar"  
}
```



19.36 JSON

DATA FORMAT

```
{  
  name: "Shariar"  
}
```

Rule 2 — String double quotes



19.36 JSON

DATA FORMAT

```
{  
  "name": "Shariar"  
}
```



19.36 JSON

DATA FORMAT

```
{  
  "name": 'Shariar'  
}
```

Rule 3 — Trailing comma নয়



19.36 JSON

DATA FORMAT

```
{  
  "name": "Shariar",  
  "age": 24,  
}
```



19.36 JSON

DATA FORMAT

```
{  
  "name": "Shariar",  
  "age": 24  
}
```

Rule 4 — Comments নেই



19.36 JSON

DATA FORMAT

```
{  
  "name": "Shariar" // user name  
}
```

Standard JSON-এ comments নেই।

19.37 JSON File

JSON আলাদা `.json` file হিসেবেও রাখা যায়।

File:

```
students.json
```

Content:

19.37 JSON

DATA FORMAT

```
{
  "students": [
    {
      "id": 1,
      "name": "Shariar",
      "department": "CSE"
    },
    {
      "id": 2,
      "name": "Rahim",
      "department": "EEE"
    }
  ]
}
```

এ ধরনের file configuration বা static data-এর জন্য ব্যবহার করা যায়।

19.38 fetch() দিয়ে JSON File পড়া

এখানে Chapter 21-এ আমরা `fetch()` বিস্তারিত শিখব; এখন শুধু JSON-এর সাথে relation দেখছি।

19.38 JAVASCRIPT

RUN

```
fetch("students.json")
  .then((response) => response.json())
  .then((data) => {
    console.log(data);
  });
```

19.38 JAVASCRIPT

RUN

```
{
  students: [
    {
      id: 1,
      name: "Shariar",
      department: "CSE"
    },
    ...
  ]
}
```

19.39 API Response হিসেবে JSON

ধরো server response:

19.39 JSON

DATA FORMAT

```
{
  "success": true,
  "message": "Login successful",
  "user": {
    "id": 101,
    "name": "Shariar",
    "role": "student"
  }
}
```

Frontend:

19.39 JAVASCRIPT

RUN

```
console.log(data.success);
```

TEXT

true

আর:

19.39 JAVASCRIPT

RUN

```
console.log(data.user.name);
```

TEXT

Shariar

19.40 REST API-তে JSON

একটি typical API request:

```
POST /api/users
```

Request body:

19.40 JSON

DATA FORMAT

```
{
  "name": "Shariar",
  "email": "shariar@gmail.com",
  "role": "student"
}
```

Server response:

19.40 JSON

DATA FORMAT

```
{
  "success": true,
  "message": "User created",
  "userId": 101
}
```

এই pattern তুমি React + Node.js/Express project-এ অনেক দেখবে।

19.41 JSON Headers

HTTP API-তে JSON পাঠানোর সময় সাধারণত:

19.41 JAVASCRIPT

RUN

```
Content-Type: application/json
```

ব্যবহার করা হয়।

JavaScript:

19.41 JAVASCRIPT

RUN

```
fetch("/api/users", {
  method: "POST",

  headers: {
    "Content-Type": "application/json",
  },

  body: JSON.stringify({
    name: "Shariar",
    email: "shariar@gmail.com",
  }),
});
```

এখানে:

19.41 JAVASCRIPT

RUN

```
body: JSON.stringify( ... )
```

খুব গুরুত্বপূর্ণ।

19.42 কেন `JSON.stringify()` দরকার?

এই object:

19.42 JAVASCRIPT

RUN

```
{
  name: "Shariar",
  age: 24
}
```

JavaScript object।

HTTP request body-তে JSON পাঠাতে:

19.42 JAVASCRIPT

RUN

```
JSON.stringify({
  name: "Shariar",
  age: 24,
});
```

হলে:

19.42 JSON

DATA FORMAT

```
{ "name": "Shariar", "age": 24 }
```

হয়।

19.43 Server থেকে JSON Response

Server response যদি JSON হয়, তখন:

19.43 JAVASCRIPT

RUN

```
const response = await fetch("/api/users");  
  
const data = await response.json();
```

এখানে:

19.43 JAVASCRIPT

RUN

```
response.json();
```

response body-এর JSON data parse করে JavaScript value দেয়।

এটি Chapter 21-এ বিস্তারিত শিখবে।

PART 05 — DEEP CLONING, SECURITY, SCHEMA & ARRAY OPERATIONS (19.44 – 19.52)

19.44 localStorage + JSON

Chapter 18-এর concept আবার:

19.44 JAVASCRIPT

RUN

```
const user = {  
  name: "Shariar",  
  role: "Developer",  
};  
  
localStorage.setItem("user", JSON.stringify(user));
```

Read:

19.44 JAVASCRIPT

RUN

```
const user = JSON.parse(localStorage.getItem("user"));
```

এখানে JSON হলো bridge:

```
JavaScript Object  
  ↓  
JSON.stringify()  
  ↓  
String  
  ↓  
localStorage
```

19.45 JSON Deep Copy Pattern

Primitive/simple JSON-compatible data-এর ক্ষেত্রে:

```
19.45 JAVASCRIPT RUN
const original = {
  name: "Shariar",
  skills: ["HTML", "CSS"],
};

const copy = JSON.parse(JSON.stringify(original));
```

এতে আলাদা object তৈরি হয়।

```
19.45 JAVASCRIPT RUN
copy.name = "Ripon";

console.log(original.name);
```

```
TEXT
Shariar
```

তবে এটি **general-purpose deep clone solution নয়**।

কারণ `Date`, `Map`, `Set`, `undefined`, functions, circular references ইত্যাদির ক্ষেত্রে JSON cloning data পরিবর্তন/হারাতে পারে বা error দিতে পারে।

Modern JavaScript-এ সাধারণ structured data clone করতে:

```
19.45 JAVASCRIPT RUN
structuredClone(original);
```

অনেক ক্ষেত্রে বেশি উপযুক্ত।

19.46 JSON Security

JSON নিজে dangerous নয়।

কিন্তু **untrusted JSON data** কীভাবে ব্যবহার করছো, সেটাই গুরুত্বপূর্ণ।

ধরো server থেকে:

```
19.46 JSON DATA FORMAT
{
  "message": "<img src=x onerror=alert('XSS')>"
}
```

তুমি যদি এটাকে:

```
19.46 JAVASCRIPT RUN
element.innerHTML = data.message;
```

দাও, security problem তৈরি হতে পারে।

বরং সাধারণ text হলে:

```
19.46 JAVASCRIPT RUN
element.textContent = data.message;
```

ব্যবহার করা safer।

19.47 JSON Schema-এর Basic Idea

Large project-এ JSON data-এর structure define করার প্রয়োজন হয়।

যেমন:

```
User
├── id → number
├── name → string
├── email → string
└── age → number
```

JSON Schema দিয়ে বলা যায়:

19.47 JSON

DATA FORMAT

```
{
  "type": "object",
  "properties": {
    "name": {
      "type": "string"
    },
    "age": {
      "type": "number"
    }
  },
  "required": ["name", "age"]
}
```

এটি data structure validation-এর জন্য ব্যবহার করা যায়।

19.48 Real-life Student API

ধরো API response:

19.48 JSON

DATA FORMAT

```
{
  "success": true,
  "students": [
    {
      "id": 1,
      "name": "Shariar",
      "department": "CSE",
      "cgpa": 3.75
    },
    {
      "id": 2,
      "name": "Rahim",
      "department": "EEE",
      "cgpa": 3.5
    }
  ]
}
```

JavaScript:

19.48 JAVASCRIPT

RUN

```
const data = {
  success: true,
  students: [
    {
      id: 1,
      name: "Shariar",
      department: "CSE",
      cgpa: 3.75,
    },
    {
      id: 2,
      name: "Rahim",
      department: "EEE",
      cgpa: 3.5,
    },
  ],
};

data.students.forEach((student) => {
  console.log(student.name, student.cgpa);
});
```

TEXT

Shariar 3.75
Rahim 3.5

19.49 JSON + `map()`

19.49 JAVASCRIPT

RUN

```
const names = data.students.map((student) => student.name);

console.log(names);
```

TEXT

["Shariar", "Rahim"]

19.50 JSON + `filter()`

CSE students:

19.50 JAVASCRIPT

RUN

```
const cseStudents = data.students.filter(
  (student) => student.department === "CSE",
);

console.log(cseStudents);
```

19.50 JAVASCRIPT

RUN

```
[
  {
    id: 1,
    name: "Sharir",
    department: "CSE",
    cgpa: 3.75,
  },
];
```

19.51 JSON + `find()`

19.51 JAVASCRIPT

RUN

```
const student = data.students.find((student) => student.id === 1);

console.log(student);
```

19.51 JAVASCRIPT

RUN

```
{
  id: 1,
  name: "Sharir",
  department: "CSE",
  cgpa: 3.75
}
```

19.52 Complete JSON Workflow

একটি real-world application:

```

  USER
  ↓
  FRONTEND
  ↓
  JavaScript Object
  ↓
  JSON.stringify()
  ↓
  HTTP Request
  ↓
  API
  ↓
  BACKEND
  ↓
  DATABASE
  ↓
  JSON Response
  ↓
  response.json()
  ↓
  JavaScript Object
  ↓
  UI

```

এটাই modern web development-এর একটি core data flow।

MUST KNOW

🔥 19.53 Must Know — JSON Syntax, Types & Limitations

এই chapter-এর সবচেয়ে গুরুত্বপূর্ণ বিষয়গুলো:

JSON Basics

```
JSON
JSON Object
JSON Array
JSON String
JSON Number
JSON Boolean
JSON null
```

Conversion

JAVASCRIPT

RUN

```
JSON.stringify();
JSON.parse();
```

Advanced

JAVASCRIPT

RUN

```
replacer
reviver
pretty printing
```

JSON Limitations

TEXT

```
No functions
No undefined
No comments
No native Date
No Map/Set
No circular references
```

Web Development

```
JSON + localStorage
JSON + fetch
JSON + REST API
JSON + HTTP
JSON + Backend
```

CHEAT SHEET

🔥 19.54 Quick Cheat Sheet — stringify, parse & Methods

কাজ	Code
Object → JSON	<code>JSON.stringify(object)</code>
Array → JSON	<code>JSON.stringify(array)</code>
JSON → Object	<code>JSON.parse(json)</code>
Pretty JSON	<code>JSON.stringify(data, null, 2)</code>
Filter properties	<code>JSON.stringify(data, ["name"])</code>
Custom stringify	<code>JSON.stringify(data, replacer)</code>
Custom parse	<code>JSON.parse(json, reviver)</code>
JSON API body	<code>JSON.stringify(data)</code>
JSON response parse	<code>response.json()</code>
localStorage save	<code>JSON.stringify(data)</code>
localStorage read	<code>JSON.parse(data)</code>

Practice 1 — Student Object

এই object তৈরি করো:

JAVASCRIPT

RUN

```
const student = {  
  name: "Shariar",  
  id: "23105101019",  
  department: "CSE",  
  semester: 8,  
  skills: ["HTML", "CSS", "JavaScript", "React"],  
};
```

এটাকে JSON string-এ convert করো।

Practice 2 — JSON → Object

এই JSON parse করো:

JSON

DATA FORMAT

```
{  
  "name": "Shariar",  
  "age": 24,  
  "role": "Full Stack Developer"  
}
```

তারপর:

JAVASCRIPT

RUN

```
console.log(user.name);  
console.log(user.role);
```

করো।

Practice 3 — Pretty JSON

Nested object তৈরি করে:

JAVASCRIPT

RUN

```
JSON.stringify(data, null, 2);
```

দিয়ে সুন্দর JSON output তৈরি করো।

Practice 4 — Product API

এই structure ব্যবহার করো:

JAVASCRIPT

RUN

```
const products = [
  {
    id: 1,
    name: "Laptop",
    price: 70000,
  },
  {
    id: 2,
    name: "Mouse",
    price: 1500,
  },
  {
    id: 3,
    name: "Keyboard",
    price: 2500,
  },
];
```

তারপর:

- JSON বানাও
- JSON parse করো
- price > 2000 filter করো
- product names বের করো

Practice 5 — localStorage

একটি user object:

JAVASCRIPT

RUN

```
{
  name: "Shariar",
  email: "shariar@gmail.com",
  role: "Developer"
}
```

`localStorage` -এ save করো এবং page reload-এর পরে আবার object হিসেবে retrieve করো।

Practice 6 — API Request Body

এই data:

JAVASCRIPT

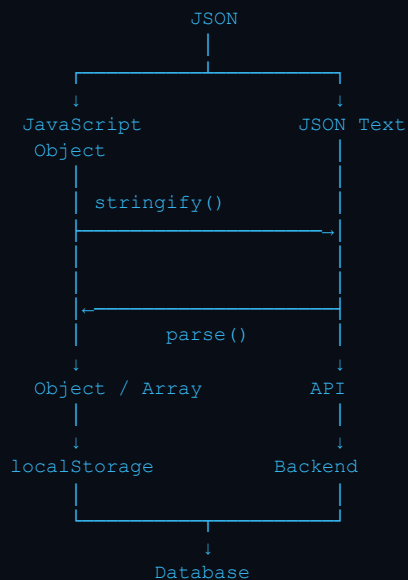
RUN

```
const user = {
  name: "Shariar",
  email: "shariar@gmail.com",
};
```

`fetch()` request-এর JSON body হিসেবে পাঠানোর code লেখো।

SUMMARY

🎯 19.56 Final Mental Map — Full-Stack JSON Workflow



★ Chapter 19-এর মূল কথা

তুমি যদি পুরো chapter থেকে শুধু এই flow-টা ভালোভাবে বুঝে ফেলো:

```

const user = {
  name: "Shariar",
  age: 24,
};

// JavaScript → JSON
const json = JSON.stringify(user);

console.log(json);

// JSON → JavaScript
const data = JSON.parse(json);

console.log(data.name);
  
```

তাহলে JSON-এর fundamental concept তোমার পরিষ্কার।

আর Full Stack Development-এ সবচেয়ে গুরুত্বপূর্ণ connection হলো:

```

JavaScript Object
  ↓
JSON.stringify()
  ↓
API Request
  ↓
Backend
  ↓
JSON Response
  ↓
response.json()
  ↓
JavaScript Object
  ↓
React UI
  
```

Chapter 19 complete.

NEXT CHAPTER

📖 Chapter 20 Preview — ES Modules (ESM)

পরবর্তী অধ্যায়ে আমরা আধুনিক মডুলার জাভাস্ক্রিপ্ট আর্কিটেকচার **ES Modules (ESM)** নিয়ে বিস্তারিত শিখব— `import`, `export`, Named Export, Default Export, Dynamic Import (`import()`), `import.meta`, এবং CommonJS বনাম ES Modules-এর পার্থক্য।